SOFTWARE ENGINEERING / TELECOMMUNICATIONS

CORE5G

Cloud-Native 5G Core Network Simulator

Cahier des charges

Version 2.0 — spécifications pour développement assisté par Codex

Langage principal :	Go
Architecture :	Microservices / gRPC
Plateforme :	Linux, Docker, Kubernetes
Statut :	Proposition à valider avant implémentation
Date :	21 octobre 2026

Résumé exécutif

Core5G est un **simulateur pédagogique inspiré du coeur de réseau 5G**, implémenté en Go. Il reproduit des parcours d'enregistrement d'équipements virtuels, d'authentification fictive, d'établissement de sessions et d'échanges de paquets simulés. Il ne constitue ni une implémentation conforme 3GPP, ni un coeur 5G de production. Les communications gRPC sont une abstraction pédagogique : les interfaces SBI du vrai 5GC suivent d'autres contrats, notamment HTTP/2 et API 3GPP. Le plan utilisateur du MVP n'implémente pas GTP-U.

Table des matières

Résumé exécutif **1**

1 Vision, objectifs et périmètre **4**

1.1 Objectifs . . . 4

1.2 MVP et exclusions . . . 4

2 Architecture et diagrammes **5**

2.1 Vue logique du simulateur . . . 5

2.2 Topologie microservices et stockage . . . 5

2.3 Séquence d'enregistrement UE . . . 6

2.4 Séquence de création d'une session PDU . . . 6

2.5 Machine à états UE . . . 6

2.6 Modèle de données . . . 7

2.7 Déploiement Docker Compose puis Kubernetes . . . 7

3 Exigences fonctionnelles détaillées **8**

3.1 F01 — Gestion des UE . . . 8

3.2 F02 — Authentification fictive . . . 8

3.3 F03 — Sessions PDU simulées . . . 8

3.4 F04 — Trafic virtuel . . . 9

3.5 F05 — Supervision et contrôle . . . 9

4 Contrats et règles d'intégration **10**

4.1 Services gRPC minimaux . . . 10

4.2 Contrat Protocol Buffers de référence . . . 10

4.3 Erreurs et délais . . . 10

5 Persistance et concurrence **12**

5.1 Modèle SQL . . . 12

5.2 Garanties . . . 12

6 Exigences non fonctionnelles **13**

7 Plan d'implémentation pour Codex **14**

7.1 Arborescence cible 14

7.2 Phases et critères de sortie 14

7.3 Règles de travail Codex 14

8 Recette et scénarios de validation **16**

9 Risques et évolutions **17**

10 Livrables finaux **18**

11 Références **19**

Table des figures

2.1 Architecture logique simplifiée : les flèches ne représentent pas des interfaces 3GPP conformes. 5
2.2 Services du MVP et dépendances d'infrastructure. Les chemins exacts seront formalisés par contrats. 5
2.3 Séquence d'enregistrement et d'authentification fictive. 6
2.4 Création d'une session : modèle logique simplifié. 6
2.5 États principaux d'un équipement UE. 6
2.6 Schéma conceptuel de persistance du MVP. 7
2.7 Topologie de déploiement cible. 7

Chapitre 1

Vision, objectifs et périmètre

1.1 Objectifs

1. Simuler les cycles de vie d'un UE et d'une session PDU.
2. Concevoir des services Go indépendants et testables.
3. Mettre en oeuvre gRPC, Protocol Buffers, PostgreSQL, Docker et l'observabilité.
4. Mesurer les performances et le comportement en cas de panne.
5. Produire un dépôt démontrable avec tests et documentation.

1.2 MVP et exclusions

MVP : UE Simulator, gNB Simulator, AMF, AUSF, UDM, SMF, UPF virtuel, API de contrôle, persistance, métriques et Docker Compose.
Hors MVP : radio 5G, véritable 5G-AKA, NAS/NGAP conformes, GTP-U, Kubernetes multi-cluster, network slicing et agent IA. Ces éléments pourront faire l'objet d'une V2.

Chapitre 2

Architecture et diagrammes

2.1 Vue logique du simulateur

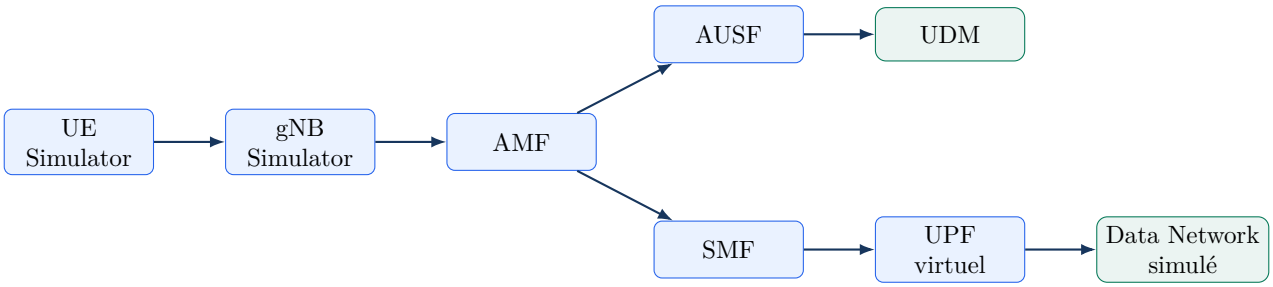


FIGURE 2.1 – Architecture logique simplifiée : les flèches ne représentent pas des interfaces 3GPP conformes.

2.2 Topologie microservices et stockage

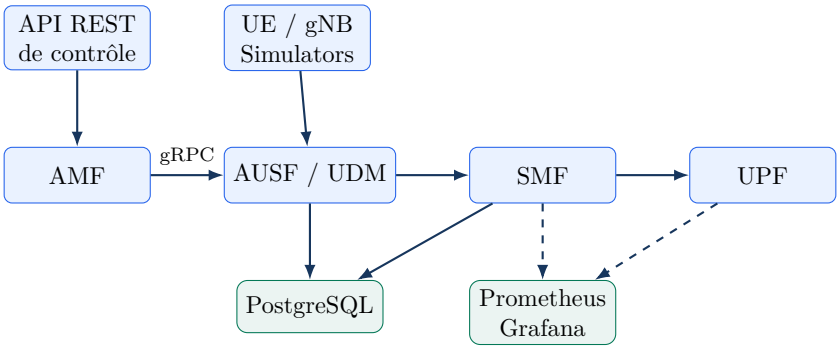


FIGURE 2.2 – Services du MVP et dépendances d'infrastructure. Les chemins exacts seront formalisés par contrats.

2.3 Séquence d'enregistrement UE

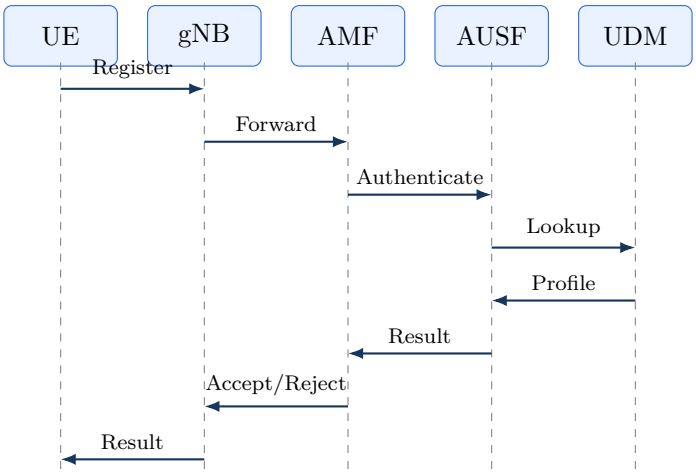


FIGURE 2.3 – Séquence d'enregistrement et d'authentification fictive.

2.4 Séquence de création d'une session PDU

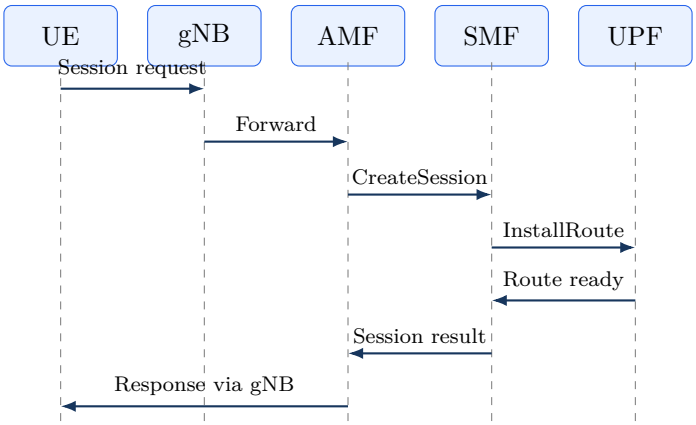


FIGURE 2.4 – Création d'une session : modèle logique simplifié.

2.5 Machine à états UE



FIGURE 2.5 – États principaux d'un équipement UE.

2.6 Modèle de données

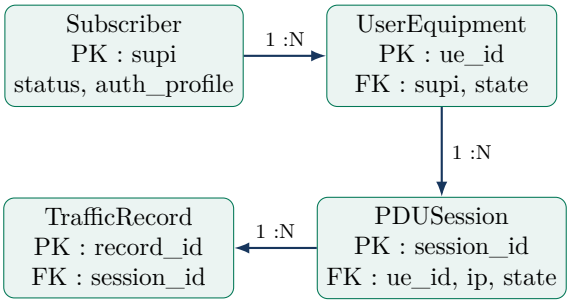


FIGURE 2.6 – Schéma conceptuel de persistance du MVP.

2.7 Déploiement Docker Compose puis Kubernetes

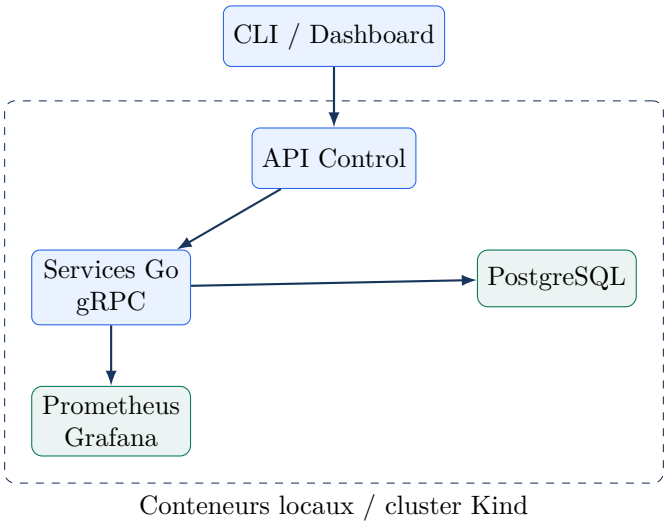


FIGURE 2.7 – Topologie de déploiement cible.

Chapitre 3

Exigences fonctionnelles détaillées

3.1 F01 — Gestion des UE

F01.1 Créer un UE avec `ue_id` UUID et `supi` fictif valide.

F01.2 Un UE ne peut pas être enregistré deux fois simultanément ; la deuxième demande reçoit un résultat idempotent ou un code d'état explicite.

F01.3 États autorisés : DISCONNECTED, REGISTERING, REGISTERED, SESSION_ACTIVE.

F01.4 En cas de timeout pendant l'enregistrement, retour à DISCONNECTED et journalisation de l'échec.

F01.5 Désenregistrement : les sessions doivent être libérées ou signalées en échec de nettoyage.

3.2 F02 — Authentification fictive

F02.1 L'AUSF vérifie l'existence et le statut du profil dans l'UDM.

F02.2 Aucun secret de production ni identifiant réel d'abonné.

F02.3 Résultats : AUTHORIZED, UNKNOWN_SUBSCRIBER, DISABLED_SUBSCRIBER, INVALID_CREDENTIALS, UNAVAILABLE.

F02.4 L'AMF ne valide jamais un UE en cas de réponse absente ou négative.

3.3 F03 — Sessions PDU simulées

F03.1 Seul un UE enregistré peut demander une session.

F03.2 Chaque session possède UUID, ue_id, adresse IP virtuelle, statut, dates et compteurs.

F03.3 Allocation IP unique parmi les sessions actives.

F03.4 Création idempotente par `request_id` ; une répétition ne crée pas de deuxième session.

F03.5 Si la configuration du UPF échoue, la session ne passe pas à ACTIVE et l'adresse IP est libérée.

F03.6 États : CREATING, ACTIVE, RELEASING, RELEASED, FAILED.

3.4 F04 — Trafic virtuel

F04.1 Un paquet possède session_id, source, destination, taille et horodatage.

F04.2 Un paquet sans session ACTIVE est rejeté.

F04.3 Simulation configurable de latence, jitter et pertes avec graine aléatoire reproductible.

F04.4 Compteurs de paquets/octets envoyés, reçus, rejetés et perdus.

3.5 F05 — Supervision et contrôle

F05.1 API REST pour lancer une simulation, consulter les UE et sessions et injecter une panne.

F05.2 Métriques Prometheus par service ; logs JSON avec trace_id et request_id.

F05.3 Health checks de disponibilité et readiness.

F05.4 Dashboard minimal : UE enregistrés, sessions actives, latence, erreurs, paquets perdus.

Chapitre 4

Contrats et règles d'intégration

4.1 Services gRPC minimaux

Service	RPC	Résultat
AMF	RegisterUE, DeregisterUE, GetUEState	Enregistrement et états
AUSF	Authenticate	Résultat d'authentification
UDM	GetSubscriber	Profil d'abonné fictif
SMF	CreateSession, GetSession, ReleaseSession	Cycle de vie session
UPF	InstallRoute, RemoveRoute, SendPacket	Routage virtuel

4.2 Contrat Protocol Buffers de référence

```
syntax = "proto3";
package core5g.amf.v1;
option go_package = "core5g/api/gen/amf/v1;amfv1";
service RegistrationService {
  rpc RegisterUE(RegisterUERequest) returns (RegisterUEResponse);
  rpc DeregisterUE(DeregisterUERequest) returns (DeregisterUEResponse);
}
message RegisterUERequest {
  string request_id = 1;
  string ue_id = 2;
  string supi = 3;
  string gnb_id = 4;
}
message RegisterUEResponse {
  bool accepted = 1;
  string registration_id = 2;
  string state = 3;
  string reason_code = 4;
}
message DeregisterUERequest { string request_id = 1; string ue_id = 2; }
message DeregisterUEResponse { bool success = 1; }
```

4.3 Erreurs et délais

Situation	Comportement exigé
Entrée invalide	INVALID_ARGUMENT, sans mutation d'état
UE ou session absent	NOT_FOUND
Transition interdite	FAILED_PRECONDITION
Service dépendant indisponible	UNAVAILABLE, échec contrôlé
Dépassement du délai	DEADLINE_EXCEEDED et compensation si nécessaire
Requête dupliquée	Réponse idempotente basée sur request_id

Délais initiaux configurables : 2 secondes par appel interservice et 5 secondes pour un parcours de bout en bout ; ce sont des valeurs de départ à ajuster après tests. Les retries ne sont permis que pour les opérations idempotentes, avec backoff borné.

Chapitre 5

Persistance et concurrence

5.1 Modèle SQL

```
CREATE TABLE subscribers (
 supi VARCHAR(32) PRIMARY KEY,
 status VARCHAR(24) NOT NULL,
 auth_profile VARCHAR(80) NOT NULL
);
CREATE TABLE user_equipments (
 ue_id UUID PRIMARY KEY,
 supi VARCHAR(32) NOT NULL REFERENCES subscribers(supi),
 gnb_id VARCHAR(64),
 state VARCHAR(24) NOT NULL,
 updated_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE TABLE pdu_sessions (
 session_id UUID PRIMARY KEY,
 request_id UUID NOT NULL UNIQUE,
 ue_id UUID NOT NULL REFERENCES user_equipments(ue_id),
 ip_address INET,
 state VARCHAR(24) NOT NULL,
 created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
 released_at TIMESTAMPTZ
);
CREATE UNIQUE INDEX uq_active_ip
 ON pdu_sessions(ip_address)
 WHERE state IN ('CREATING','ACTIVE','RELEASING')
 AND ip_address IS NOT NULL;
```

5.2 Garanties

- Les transitions d'état sont atomiques avec contrôle de version ou verrou transactionnel.
- Les opérations concurrentes sur un même UE sont sérialisées logiquement.
- Les adresses IP libérées peuvent être réutilisées uniquement après nettoyage confirmé.
- Les services ne partagent pas directement leurs tables métier en architecture cible ; le MVP peut utiliser une instance PostgreSQL avec schémas séparés.
- Les traces de trafic agrégées ne doivent pas saturer la base transactionnelle.

Chapitre 6

Exigences non fonctionnelles

ID	Exigence	Vérification
NF01	1000 UE concurrents en objectif de charge	Scénario reproductible
NF02	p95 enregistrement local inférieur à 100 ms (cible, non garantie)	Rapport de benchmark
NF03	Pas de crash lors des pannes simulées	Tests de résilience
NF04	80% de couverture des packages métier critiques	go test -cover
NF05	Aucun data race connu	go test -race
NF06	Démarrage Docker Compose en une commande	Test d'installation
NF07	Métriques, logs structurés, traces	Vérification Grafana
NF08	Aucun secret réel commité	CI et revue de code

Les résultats dépendent du matériel ; documenter CPU, RAM, OS, nombre de connexions, profil de charge et nombre d'itérations.

Chapitre 7

Plan d'implémentation pour Codex

7.1 Arborescence cible

```
core5g/
  cmd/{amf,ausf,udm,smf,upf,ue-simulator,gnb-simulator,control-api}/
  internal/{registration,authentication,session,routing,subscriber}/
  api/proto/
  api/gen/
  pkg/{config,logging,telemetry}/
  migrations/
  deployments/{docker,kubernetes}/
  tests/{integration,e2e,performance}/
  docs/
  .github/workflows/
  docker-compose.yml
  Makefile
  go.mod
  README.md
```

7.2 Phases et critères de sortie

Phase	Livrable	Critère de sortie
1	Dépôt, Makefile, CI, protobuf, Docker	Build et tests de base OK
2	UDM, AUSF, AMF, UE/gNB	Enregistrement nominal et refusé OK
3	SMF, UPF virtuel, IPAM	Session et trafic virtuel OK
4	Monitoring, scénarios de panne	Métriques et erreurs observables
5	Kubernetes Kind, charge, documentation	Démo reproductible et rapport

7.3 Règles de travail Codex

1. Lire le présent document avant toute modification.
2. Produire un plan et signaler les ambiguïtés avant de coder.
3. Une fonctionnalité par branche `feature/...` ; ne pas modifier `main` directement.

4. Écrire les tests de chaque fonctionnalité et exécuter `go test ./...` et `go vet ./...`.
5. Ne pas annoncer une fonctionnalité comme terminée sans preuve de test.
6. Maintenir README, ADR et documentation API à jour.
7. Ne jamais inventer de résultats de benchmark.

Chapitre 8

Recette et scénarios de validation

ID	Action	Résultat attendu
T01	Enregistrer un abonné connu	REGISTERED
T02	Enregistrer un SUPI inconnu	Refus, aucun état REGISTERED
T03	Répéter la même requête	Pas de duplication
T04	Créer une session sur UE enregistré	ACTIVE et IP unique
T05	Créer une session sur UE non enregistré	FAILED_PRECONDITION
T06	Envoyer paquet sur session ACTIVE	Compteurs mis à jour
T07	Envoyer paquet sur session RELEASED	Rejet contrôlé
T08	Couper le UPF pendant création	Échec et compensation
T09	Libérer session deux fois	Idempotence
T10	Lancer 1000 UE	Rapport p50/p95/p99, erreurs, CPU/RAM

Chapitre 9

Risques et évolutions

Risque	Mitigation
Complexité 3GPP	Distinguer simulation pédagogique et conformité réelle
États incohérents	Transitions atomiques, idempotence, compensation
Couplage excessif	Contrats protobuf et ownership des données
Charge et latence	Benchmarks progressifs et profiling Go
Complexité Kubernetes	Valider d'abord Docker Compose

V2 possibles : interfaces SBI inspirées des spécifications 3GPP, GTP-U expérimental, network slicing simplifié, module C++ de traitement des paquets, agent IA de diagnostic, multi-cluster Kubernetes.

Chapitre 10

Livrables finaux

— Dépôt GitHub avec historique de branches et Pull Requests.
— Code Go, fichiers proto et scripts de génération.
— Migrations PostgreSQL, Docker Compose, manifests Kind/Kubernetes.
— Tests unitaires, intégration, E2E, charge et résilience.
— Dashboard de métriques et rapport de performances.
— Documentation d'architecture, API, décisions et guide de démonstration.
— Captures ou vidéo d'une exécution réelle.

Chapitre 11

Références

— 3GPP TS 23.501 — System Architecture for the 5G System.
— 3GPP TS 23.502 — Procedures for the 5G System.
— 3GPP TS 29.500 — Service Based Architecture.
— https://grpc.io/docs/
— https://go.dev/doc/
— https://kubernetes.io/docs/
— https://open5gs.org/
— https://free5gc.org/